

--{ python для пентестера }--

Ethical Hacking Tutorial Group The Codeby

представляет



PYTHON

«Для Пентестера»

Programming Language..



-- { ПРОДВИНУТЫЙ УРОВЕНЬ } --

Модуль queue

Модуль queue реализует механизм очереди. Он довольно часто применяется в многопоточном программировании, когда необходимо безопасно передавать информацию между потоками. Класс Queue этого модуля реализует все необходимые блокировки.

Модуль предоставляет реализации трех типов очередей, единственная разница которых это порядок получаемых значений.

Класс `queue.Queue(maxsize=0)`

Очередь типа FIFO (First Input First Output - первым пришел, первым вышел).

`maxsize` - это целое число, устанавливающее максимальное количество элементов, которые могут быть помещены в очередь. Вставка новых элементов блокируется после достижения этого размера, пока элементы очереди не будут использованы. Если `maxsize` меньше или равен нулю, размер очереди бесконечен.

Пример FIFO:

```
from queue import Queue

q = Queue()
for i in range(1, 6):
    q.put(i)

while not q.empty():
    print(q.get(), end=' ')

my_queue x
1 2 3 4 5
```

Метод `Queue.put(item, block=True, timeout=None)`

Помещает объект `item` в очередь. Если `block=True` и `timeout` не задан (`None` по умолчанию), то при необходимости произойдет блокировка до тех пор пока в очереди не будет доступного места. Если `timeout` это положительное число, то произойдет блокировка на заданное число секунд и будет возбуждено исключение `Queue.Full` если в течении этого времени не освободится место в очереди. В другом случае (если `block=False`), метод помещает объект `item` в очередь если свободное место доступно немедленно, иначе генерирует исключение `Queue.Full` (параметр `timeout` в этом случае игнорируется).

Метод `Queue.put_nowait(item)`

Эквивалентен вызову `put(item, False)`

+-----+

Метод `Queue.empty()`

+-----+

Возвращает `True` если очередь пуста, `False` в противном случае. Однако если метод возвращает `True`, то не гарантируется что следующий вызов `put()` не будет блокирован. Также если возвращает `False`, не гарантируется что последующий `get()` не будет блокирован.

+-----+

Метод `Queue.get(block=True, timeout=None)`

+-----+

Удалить и вернуть элемент из очереди. Если `block=True` и `timeout` не задан (`None` по умолчанию), произойдет блокировка до тех пор пока элемент не будет доступен. Если `timeout` это положительное число, то произойдет блокировка на заданное число секунд и будет возбуждено исключение `Queue.Empty` если в течении этого времени элемент не будет доступен. В другом случае (если `block=False`), метод возвращает элемент очереди немедленно, иначе генерирует исключение `Queue.Empty` (параметр `timeout` в этом случае игнорируется).

+-----+

Метод `Queue.qsize()`

+-----+

Возвращает примерный (аппроксимированный) размер очереди. Обратите внимание, что `qsize() > 0` не гарантирует, что последующий `get()` не будет блокироваться, и `qsize() < maxsize` не гарантирует, что `put()` не будет блокироваться.

+-----+

Метод `Queue.task_done()`

+-----+

Сообщает что ранее полученное задание из очереди выполнено. Используется потребительскими потоками. Для каждого вызова `get()` последующий вызов `task_done()` сообщает объекту очереди что обработка задания завершена.

+-----+

Метод `Queue.join()`

+-----+

Блокирует, пока все элементы из очереди не будут получены и обработаны. Количество незавершенных задач увеличивается при каждом добавлении элемента в очередь. Счетчик сбрасывается всякий раз, когда потребительский поток вызывает `task_done()` чтобы указать, что элемент был получен и вся работа над ним завершена. Когда количество незавершенных задач падает до нуля, `join()` разблокируется.

Пример FIFO с методом qsize():

```
from queue import Queue

q = Queue()
for i in range(1, 6):
    q.put(i)

for i in range(q.qsize()):
    print(q.get(), end=' ')

q_size x
1 2 3 4 5
```

Пример проверки пуста ли очередь:

```
from queue import Queue

q = Queue()
for i in range(1, 6):
    q.put(i)

print(q.empty()) # Очередь НЕ пуста
while not q.empty():
    print(q.get(), end=' ')
print('\n' + str(q.empty())) # Очередь пуста

tr x
False
1 2 3 4 5
True
```

Класс `queue.LifoQueue(maxsize=0)`

Очередь LIFO, или по другому "стэк" (Last Input First Output - последним пришел, первым вышел).

Пример LIFO:

```
from queue import LifoQueue

q = LifoQueue()
for i in range(1, 6):
    q.put(i)

while not q.empty():
    print(q.get(), end=' ')

tr x
5 4 3 2 1
```

Класс `queue.PriorityQueue(maxsize=0)`

Этот класс реализует очередь с приоритетами элементов. То есть элементы с более высоким приоритетом будут выходить в первую очередь вне зависимости от того, когда пришли.

Пример Priority:

```
from queue import PriorityQueue

q = PriorityQueue()
q.put((2, "a"))
q.put((0, "b"))
q.put((1, "c"))
q.put((8, "d"))
q.put((1, "e"))
q.put((4, "f"))
while not q.empty():
    print(q.get())
```

my_queue x

```
(0, 'b')
(1, 'c')
(1, 'e')
(2, 'a')
(4, 'f')
(8, 'd')
```

Стоит отметить что поддерживаются отрицательные числа, и так как они меньше, то приоритет будет более высокий.

my_queue x

```
(-10, 'c')
(-1, 'b')
(1, 'e')
(2, 'a')
(4, 'f')
(8, 'd')
```

Process finished with exit code 0

Пример работы очередей с использованием многопоточности:

```
from queue import LifoQueue
from threading import Thread
from time import sleep

q = LifoQueue()

# Функция генерирующая данные для очереди
def source(n):
    return [i for i in range(1, 1 + n)]

# Функция заполняющая очередь заданиями
def put(n):
    for item in source(n):
        q.put(item)

def worker():
    while True:
        # Получаем задание из очереди
        item = q.get()
        print(f'Очередь: {item} выполняется')
        # Сообщаем о выполненном задании
        q.task_done()
        print(f'Очередь: {item} завершилась\n')

# Создаем и запускаем потоки, которые будут обслуживать очередь
for x in range(1, 4):
    t = Thread(target=worker)
    t.daemon = True
    t.start()
    print('Поток', x, 'стартовал')
    put(x)
    sleep(2)
```

```
main_q x
Поток 1 стартовал
Очередь: 1 выполняется
Очередь: 1 завершилась

Поток 2 стартовал
Очередь: 2 выполняется
Очередь: 2 завершилась

Очередь: 1 выполняется
Очередь: 1 завершилась

Поток 3 стартовал
Очередь: 3 выполняется
Очередь: 3 завершилась

Очередь: 2 выполняется
Очередь: 2 завершилась

Очередь: 1 выполняется
Очередь: 1 завершилась
```

В этом коде использовался демонический поток.

Поток демона - это поток, который не будет продолжать работать, если основной поток остановился, и больше не осталось потоков, не являющихся демонами. Основной поток тоже не является демоном.

Это означает, что установка значения демона потока в `True` будет говорить, что он не будет продолжать работать после завершения основного потока (или других потоков, не являющихся демонами). Мы можем установить флаг `daemon` в `False` чтобы поток продолжал работать, даже когда основной поток завершает работу. **Значение демона должно быть установлено до `start()` вызова в потоке.**

Говоря по-другому настройка флага `thread.daemon = True` позволит основной программе выйти. Именно поэтому бесконечный цикл прервётся, хотя его прерывание не было написано явным образом.

Модуль `multiprocessing`

В большинстве случаев, многопоточность кода написанного с помощью модуля `threading` не заставит его работать быстрее. Потоки Python предназначены для одновременного выполнения нескольких задач, однако это происходит только на одном ядре ЦП.

А чтобы несколько задач выполнялись на нескольких ядрах процессора, нужно использовать модуль `multiprocessing`. С помощью этого модуля мы можем запускать множество процессов на нескольких ядрах ЦП. И каждый процесс выполняется независимо от других, этот метод параллелизма позволяет избежать проблем с GIL.

Класс `Process`

Создание процесса происходит путем создания объекта класса `Process`. Аргументы аналогичны тем, что есть в классе `Thread`.

Пример создания процесса:

```
from multiprocessing import Process

def proc(name):
    print('Hello', name)

p = Process(target=proc, args=('World!'))
p.start()
p.join()
```

mult x

Hello World!

Таким образом созданный процесс работает параллельно основному потоку, а не внутри его.

Возьмём код со стрелами, который у нас был в предыдущем уроке, и подправим для работы с процессами. И чтобы увидеть разницу, идентифицируем каждую стрелу.

Отличий совсем немного внешне. Однако при запуске мы можем видеть что порядок запуска стрел у нас случайный. Если нумерацию добавить к коду с модулем `threading`, то там всегда будет идти запуск потоков в порядке очереди.

```
from multiprocessing import Process
import time

start = time.time()
def sec(x):
    print(f'Стрела {i+1} полетела...')
    time.sleep(x)
    print(f'Стрела {i+1} поразила цель!')

my_list = []
for i in range(3):
    mult = Process(target=sec, args=(5,))
    my_list.append(mult)
    mult.start()

for my_thread in my_list:
    my_thread.join()
print(f'Completed in: {(time.time() - start):.3f} sec.')
```

```
multi x
Стрела 3 полетела...
Стрела 1 полетела...
Стрела 2 полетела...
Стрела 3 поразила цель!
Стрела 2 поразила цель!
Стрела 1 поразила цель!
Completed in: 5.008 sec.
```

Кроме того что запуск случайный, так и до цели стрелы прилетают тоже не строго по-очереди. Таким образом совершенно очевидно, что процессы не зависят друг от друга, и идут параллельно. Более наглядно наблюдать это с большим количеством стрел, например 8-9.

Класс `Queue`

Этот класс является практически клоном `queue.Queue`. Упорядочим полёт стрел.

```
from multiprocessing import Process, Queue
import time

start = time.time()
def sec(x, y):
    y.put(i)
    print(f'Стрела {i+1} полетела...')
    time.sleep(x)
    print(f'Стрела {i+1} поразила цель!')

q = Queue()
my_list = []
for i in range(5):
    mult = Process(target=sec, args=(5, q))
    my_list.append(mult)
    mult.start()
    q.get()

for my_thread in my_list:
    my_thread.join()
print(f'Completed in: {(time.time() - start):.3f} sec.')
```

Теперь, применив очередность FIFO с помощью `Queue`, мы увидим на выходе следующий результат:

```
my_mult x
Стрела 1 полетела...
Стрела 2 полетела...
Стрела 3 полетела...
Стрела 4 полетела...
Стрела 5 полетела...
Стрела 1 поразила цель!
Стрела 2 поразила цель!
Стрела 3 поразила цель!
Стрела 4 поразила цель!
Стрела 5 поразила цель!
Completed in: 5.026 sec.
```

Класс `Lock`

Модуль `multiprocessing` содержит эквиваленты всех примитивов синхронизации из `threading`. Например, можно использовать блокировку, чтобы гарантировать, что только один процесс печатает на стандартный вывод в один момент времени.

Пример из документации просто печатающий фразу с порядковым номером:

```
from multiprocessing import Process, Lock

def f(l, i):
    l.acquire()
    try:
        print('hello world', i)
    finally:
        l.release()

if __name__ == '__main__':
    lock = Lock()

    for num in range(10):
        Process(target=f, args=(lock, num)).start()

q_mult x
hello world 0
hello world 5
hello world 7
hello world 6
```

Давайте убедимся, что без замков возникнут проблемы, и уберём их. А заодно увеличим количество итераций до 1000. Вот какие накладки мы получим на выходе:

```
hello world 496
hello world 497
hello worldhello world 499
498
hello world 500
```

Сам код без замков:

```
from multiprocessing import Process

def f(i):
    print('hello world', i)

for num in range(1000):
    Process(target=f, args=(num,)).start()
```

Класс **Pool**

Это удобное средство распараллеливания выполнения функции по нескольким входным значениям, распределения входных данных по процессам.

Чтобы создать несколько процессов нет необходимости использовать цикл. Для этого достаточно указать в скобках количество процессов **Pool(5)**. Также можно писать опциональный аргумент **Pool(processes=5)**.

Наиболее применяемые функции: **Pool.apply**, **Pool.map**. Функции **map** и **apply** - блокирующие вызовы. Главный поток будет заблокирован, пока процесс не выполнит работу.

В Python встроенная **функция map** принимает два аргумента: функцию и аргумент составного типа данных, например, список. И **map** применяет к каждому элементу списка переданную функцию.

Класс **Pool** поддерживает менеджера контекста **with**. Так запись запуска пула процессов вида:

```
if __name__ == '__main__':
    p = Pool(5)
    print(p.map(func, [1, 2, 3]))
```

Будет полностью аналогичной записи вида:

```
if __name__ == '__main__':
    with Pool(5) as p:
        print(p.map(func, [1, 2, 3]))
```

Пример с обоими функциями **map** и **apply**, делающими в данном коде одну и ту же работу:

```
from multiprocessing import Pool

def func(x):
    return x**3

if __name__ == "__main__":
    pool = Pool(5) # создаем пул из 5 процессов
    # раскидываем числа от 1 до 6 по 5 процессам
    results = [pool.apply(func, args=(x,)) for x in range(1,7)]
    print(results)

    pool = Pool(5)
    # раскидываем числа от 1 до 6 по 5 процессам
    results = pool.map(func, range(1,7))
    print(results)
```

my_pool x

```
[1, 8, 27, 64, 125, 216]
[1, 8, 27, 64, 125, 216]
```